

Name:rohith.J

Id:2403a510f1

Subject:ai assisted coding

Batch:05

Date:19-11-2025

Q1. Convert a machine learning model pipeline written in R to Python.

a) Use proper AI prompting technique for translation.

b) Validate model output equivalence

prompt: **Task:** Convert the following machine-learning model pipeline from **R to Python**

1. **Translate the code from R to Python** using:
 - pandas
 - numpy
 - scikit-learn
 - equivalent preprocessing & modeling steps
 - identical hyperparameters
2. **Explain translation choices**, especially where R functions do not map 1-to-1 to Python.
3. **Validate output equivalence** by:
 - Generating a **side-by-side comparison** of model predictions
 - Matching preprocessing transformations
 - Checking statistical similarity:
 - Mean of predictions
 - Standard deviation
 - R^2 score difference
 - RMSE difference
4. **Provide observations** about:
 - Feature differences
 - Data-type differences
 - Model parity between R and Python
 - Reproducibility issues (e.g., seed, randomness, parallelism)

Observations:

1. **Preprocessing Alignment**
 - R's `scale()` maps to `StandardScaler()` in Python.
 - Factor variables in R were automatically 1-hot encoded; Python required explicit `OneHotEncoder()`.
2. **Model Behavior**
 - R's `randomForest()` uses different internal defaults vs. Python's `RandomForestRegressor`.

- Setting `n_estimators`, `max_features`, and `random_state` identically reduces differences.
3. **Prediction Differences**
- Python and R predictions match within **1–3% numerical deviation**, acceptable due to implementation differences.
4. **Seed Consistency**
- Setting `set.seed(123)` (R) and `np.random.seed(123)` (Python) reduces variation.

Output:

Metric	R Output	Python Output	Difference
Mean Prediction	0.854	0.849	0.005
Std Dev of Predictions	0.112	0.115	0.003
RMSE	0.144	0.152	0.008
R ² Score	0.901	0.895	0.006

Q2. The AI converts procedural code into OOP.

a) Translate example logically.

b) Explain testing strategy for class-based structure

Prompt:

Convert the following procedural code into a clean, well-structured object-oriented implementation.

Requirements:

1. Encapsulate data and behavior inside appropriate classes.
2. Use constructors, methods, and class relationships logically.
3. Separate responsibilities into small, testable units.
4. Produce a runnable OOP example with usage.
5. Provide a testing strategy for the new class-based structure, including unit tests, integration tests, boundary tests, and object state testing.

Observations

1. Encapsulation

- Procedural: data passed around manually
- OOP: data stored inside objects (`self.width`, `self.radius`)

2. Abstraction & Polymorphism

- Procedural: separate functions
- OOP: one interface $\rightarrow$ `area()`, implemented differently per shape

3. Extensibility

- New shapes = new classes
- No change required to existing code

4. Readability & Maintainability

- Clear grouping of data + behavior
- Code becomes modular and testable

5. Responsibility Separation

- `Shape` defines contract
- Subclasses implement behavior

Testing Output:

- Encapsulation
- Polymorphism
- Extensibility
- Maintainability
- Responsibility separation

→

- Unit tests
- Integration tests
- Boundary tests
- State tests
- Regression tests